Techfest 2026-27

# ContextCon IIT Bombay: The Agentic Services Hackathon

Sponsored by:

CrustData Y Combinator

**SUBMISSION:**

THEME:

Hiring: Recruiters and staffing agencies paid on outcomes, finding candidates who fit and verifying their career history.

TEAM:

CandiQ

*Candidate intelligence*


1. Kusumuru Suresh Kumar 26M2029
✉ 26m2029@iitb.ac.in
✉ kusumurusureshkumar21@gmail.com
📞 9182592794
M. Tech 1st year, Systems and Control Engineering, IIT Bombay

2. Karan Ahirwar 26M2022
✉ karan.ahirwar@iitb.ac.in
✉ karanahirwar32128@gmail.com
📞 9981789428
M. Tech 1st year, Systems and Control Engineering, IIT Bombay


Find this at:
https://github.com/suresh-vlsi/CandiQ
https://www.loom.com/share/9a460cf12f0c4484b9deeab9a9d10283

# EVIDENCE-BASED AI RECRUITMENT AGENT

*A human-in-the-loop recruiting research assistant*

Implementation, Requirements, Execution Guide, Limitations and Future Scope

| | |
|---|---|
| Implementation environment: | Ubuntu on WSL · Python 3.14.4 · VS Code |
| Data source: | Crustdata Person Search and Enrichment API |
| LLM provider: | Groq via OpenAI-compatible API |
| Assessment example: | Senior ASIC Physical Design Engineer · Bengaluru, India |

*Documentation date: 31 October 2026*

## 1. Project overview:

The Evidence-Based AI Recruitment Agent is a human-in-the-loop recruiting research assistant. It converts a recruiter's natural-language hiring request into structured criteria, searches candidate profiles through Crustdata, enriches selected profiles, extracts evidence-linked claims, verifies those claims against source records, calculates transparent match scores, and produces a Markdown shortlist for recruiter review.

The project is designed to reduce repetitive candidate-research work while making the reasoning behind a shortlist inspectable. It is not an autonomous hiring system: it does not contact candidates, schedule interviews, make offers, or make employment decisions.

| | |
|---|---|
| Input | Natural-language role description and must-have / nice-to-have requirements. |
| Candidate data | Crustdata Person Search and Person Enrichment API; mock data fallback is available. |
| Language model | Groq model accessed through an OpenAI-compatible chat-completions endpoint. |
| Processing | Python pipeline, Pydantic data models, deterministic evidence and scoring logic. |
| Output | Markdown candidate shortlist with criterion-level evidence, scores, coverage, and review notes. |
| Operating mode | Local execution from Ubuntu on Windows Subsystem for Linux (WSL). |

## 2. What we are implementing:

The implementation is an end-to-end candidate discovery and evidence assessment workflow. The language model assists with interpretation and claim generation; deterministic Python code checks source-field paths, applies domain-specific skill rules, derives timeline information, and calculates scores.

### 2.1 End-to-end workflow:

i. Parse recruiter request — The LLM converts the request into a role title, atomic criteria, priorities, categories, assumptions, possible conflicts, and search variants.
ii. Generate search variants — Strict, relaxed, and adjacent title variants broaden discovery while retaining the original hiring intent.
iii. Search candidate profiles — The Crustdata client submits filters, follows pagination cursors, and retrieves profile summaries.
iv. Deduplicate and pre-rank — Candidates are deduplicated using Crustdata person IDs and ranked using available profile text before expensive enrichment.
v. Enrich selected profiles — Professional profile URLs are submitted to Crustdata enrichment. The client selects the best match only when its confidence score is at least 0.8.
vi. Derive structured facts — Python computes experience summaries and timeline-related indicators from employment records; these are not invented by the LLM.

vii. Generate atomic claims — The LLM proposes one checkable claim at a time and associates it with a record ID, field path, and source value.
viii. Validate and verify claims — The system checks that cited paths and values exist in the actual candidate record, applies deterministic skill/seniority guards, and uses a verifier to classify claims.
ix. Score candidates — The scoring layer combines criterion priorities and verdicts, preserves unknown states, checks must-have contradictions, and reports evidence coverage.
x. Generate report — The pipeline writes a Markdown shortlist containing candidate scores, criterion tables, supporting evidence, rejected claims, and review flags.

2.2 Architecture and module responsibilities:

| | |
|---|---|
| pipeline.py | Orchestrates request parsing, candidate search, pre-ranking, enrichment, claims, verification, scoring, and report generation. |
| crustdata.py | Calls Crustdata search/enrichment endpoints; handles request construction, pagination, caching, throttling, retries, and confidence filtering. |
| llm.py | Wraps the configured LLM provider; parses hiring requests, generates claims, and performs claim verification. |
| prompts.py | Stores prompts and instructions for request parsing, claim generation, and verification. |
| schemas.py | Defines Pydantic models for criteria, search variants, evidence, claims, verdicts, and candidate reports. |
| derived.py | Computes deterministic derived profile information, including experience/timeline summaries. |
| mock_data.py | Provides mock candidate data and compatible search/enrichment functions for development or fallback. |
| tests/ | Automated tests for evidence validation, scoring regressions, skill taxonomy, and related logic. |
| .env | Local configuration for API keys, provider URL, model name, and runtime settings; must remain private. |
| .cache/ and .llm_cache/ | Local response caches used to reduce repeated external requests and LLM calls. |

3. Requirements:

3.1 Functional requirements:

- Accept a recruiter's natural-language request rather than requiring a rigid search form.
- Extract atomic requirements and distinguish must-have from nice-to-have criteria.
- Generate strict, relaxed, and adjacent search variants.
- Search candidate profiles and enrich only a selected subset to control latency and API usage.
- Retain source record IDs and exact nested field paths for candidate evidence.
- Reject claims whose cited field path or quoted value cannot be validated.
- Distinguish supported, partially supported, unsupported, and contradicted claims.
- Represent criterion outcomes as yes, partial, no, or unknown.
- Calculate a weighted match score and a separate evidence-coverage measure.
- Require human review when must-have evidence is unknown or contradictory.
- Produce a readable Markdown shortlist and retain rejected claims for auditability.

3.2 Software and service requirements

| Requirement | Current project configuration / notes |
|---|---|
| Operating system | Windows 11 with Ubuntu running through WSL; commands below assume the Ubuntu shell. |
| Editor | Visual Studio Code, preferably opened in the WSL project directory. |
| Python | Python 3.14.4 was observed in the active virtual environment during validation. |
| Virtual environment | A project-local .venv directory. |
| Python dependencies | Install the project's pinned/current dependency declarations from requirements.txt using pip. |
| Candidate data API | Crustdata API key with access to Person Search and Person Enrichment. |

| | |
|---|---|
| LLM API | Groq API key and a model available to the account; configured through the OpenAI-compatible endpoint. |
| Internet | Required for live Crustdata and Groq API calls; cached responses may reduce repeated calls. |
| Testing | pytest; the recorded validation environment used pytest 9.2.0. |

The repository's requirements.txt is the authoritative dependency list. The setup process installs dependencies from that file rather than relying on manually installed global packages.

3.3 Environment variables and secrets

| Variable | Purpose |
|---|---|
| CRUSTDATA_API_KEY | Authenticates live candidate search and profile enrichment. |
| LLM_API_KEY | Authenticates the configured LLM provider (Groq in the demonstrated run). |
| LLM_BASE_URL | OpenAI-compatible provider endpoint; Groq endpoint used: https://api.groq.com/openai/v1. |
| LLM_MODEL | Model identifier configured for the Groq account; demonstrated model: openai/gpt-oss-20b. |
| CRUSTDATA_BASE_URL | Optional override for the Crustdata API base URL. |
| CRUSTDATA_API_VERSION | Optional API version header override. |
| MIN_CANDIDATES / SEARCH_LIMIT / MAX_SEARCH_PAGES | Search breadth and pagination controls. |
| ENRICH_BUFFER | Number of extra candidates beyond --top to enrich; setting it to 0 limits enrichment to --top. |
| LLM_MAX_COMPLETION_TOKENS | Optional response-token budget used in the Groq configuration. |

4. Installation and setup commands:

Open the Ubuntu/WSL terminal in VS Code and navigate to the project directory. Example path used in the implementation:

```
cd /mnt/c/Users/Lenovo/CrustData_Hackathon
```

Create and activate the virtual environment (skip creation if .venv already exists):

```
python3 -m venv .venv
source .venv/bin/activate
```

Install or refresh dependencies:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Create the local environment file from the example, if not already created:

```
cp .env.example .env
```

*Edit .env locally and set the Crustdata and Groq configuration. Do not paste the secret values into documentation.*

Load .env values into the current WSL terminal session:

```
set -a
source .env
set +a
```

Optional environment diagnostics without printing secrets:

```
python -c "import os; print('CRUSTDATA_API_KEY:',
bool(os.getenv('CRUSTDATA_API_KEY'))); print('LLM_API_KEY:',
```

```
bool(os.getenv('LLM_API_KEY'))); print('LLM_BASE_URL:',
os.getenv('LLM_BASE_URL')); print('LLM_MODEL:', os.getenv('LLM_MODEL'))"
```

5. Commands used during development and execution:

5.1 Run the recruitment pipeline

The demonstrated hiring request was for a senior ASIC Physical Design Engineer in Bengaluru, India:

```
python pipeline.py "Find a senior ASIC Physical Design Engineer in Bengaluru,
India. Must-have: professional experience in ASIC physical design. Nice-to-have:
experience with Synopsys tools, timing analysis, floorplanning, placement and
routing." --top 5 --out final_recruitment_report.md
```

Meaning of the main arguments:

| Argument | Meaning |
|---|---|
| pipeline.py | Entry-point script that runs the full pipeline. |
| Quoted request | Natural-language role and requirements supplied to the parser. |
| --top 5 | Requests a final shortlist of up to five candidates. |
| --out final_recruitment_report.md | Writes the Markdown report to the specified file. |
| --outreach | Optional flag for locally generated outreach drafts; drafts are not sent. |

5.2 Syntax and automated tests

```
python -m compileall -q .
python -m pytest tests/ -v
```

The recorded final test run collected and passed 23 tests. The suite includes core evidence-path validation, scoring regression tests, and semiconductor skill-taxonomy tests. These tests run locally and do not require live candidate API calls.

5.3 Useful inspection and Git commands

```
pwd
ls -la
find . -maxdepth 2 -type f
cat requirements.txt
git status --short
```

Git workflow after reviewing generated files and confirming that no secrets or candidate exports are staged:

```
git add README.md *.py tests/ requirements.txt .env.example
git status --short
git commit -m "Document evidence-based AI recruitment agent"
git push
```

*Do not blindly use git add . when the working directory may contain .env, caches, virtual environments, or real candidate data. Inspect git status first and confirm .gitignore covers sensitive files.*

6. Candidate matching and scoring methodology:

6.1 Evidence model

A candidate claim is not accepted merely because it sounds plausible. Each claim is expected to carry a source record identifier, an exact field path, and the value copied from that field. The implementation validates the path and value against the candidate record before accepting the claim. A separate verification stage assesses whether the evidence actually supports the wording.

| Verdict / state | Interpretation |
|---|---|
| Supported | The cited source evidence supports the claim. |
| Partial | The source supports a weaker or narrower version; a caveat or corrected wording is retained. |

| | |
|---|---|
| Unsupported | The cited evidence does not establish the claim. |
| Contradicted | The evidence indicates the opposite of the claim. |
| Unknown | There is not enough qualifying evidence to determine the criterion. |

6.2 Semiconductor skill taxonomy

Deterministic rules reduce common vendor/tool conflation for the ASIC physical-design use case:

| Observed skill | What it can support | What it does not prove |
|---|---|---|
| PrimeTime / Primetime | Synopsys tool experience and timing-analysis experience. | Floorplanning, placement, or routing experience. |
| STA | Timing-analysis discipline. | Synopsys vendor/tool experience by itself. |
| Innovus | Cadence tool experience (where relevant to the taxonomy). | Synopsys tool experience. |
| Place & Route | Placement and routing criteria. | Floorplanning by itself. |
| Floorplanning | Floorplanning criterion. | Placement/routing unless separately evidenced. |

6.3 Score and evidence coverage

- The current scoring configuration weights must-have criteria at 3 points and nice-to-have criteria at 1 point. Yes receives full credit, partial receives half credit, and no/unknown receive no score credit. The system also reports evidence coverage so a high score is not mistaken for complete evidence.
- Unknown is not equivalent to no: missing evidence is not proof that a candidate lacks a skill.
- An unknown must-have can trigger needs_review.
- A verifier-confirmed contradiction on a must-have can result in not_recommended.
- A high match score with low evidence coverage must be interpreted cautiously.
- Employment timeline anomalies are screening prompts to verify, not conclusions about a person.

7. Demonstrated execution and validation:

In the final demonstrated live run, Crustdata search returned five candidates for the strict search variant, five profiles were enriched successfully, and five candidates were analysed. The pipeline generated 28 claims, of which six were rejected by the verifier. A corrected shortlist was then prepared by reconciling those claims with the source evidence; no further LLM analysis was needed for that report.

| Observed item | Recorded result |
|---|---|
| Live data source | Crustdata API |
| Search variant | Strict |
| Candidates found | 5 |
| Profiles enriched | 5 |
| Candidates analysed | 5 |
| Claims generated | 28 |
| Claims rejected by verifier | 6 |
| Automated tests | 23 passed in the recorded run |
| Final output | Markdown shortlist; corrected PDF/DOCX versions were also prepared. |

These figures describe the demonstrated run only. They are not a guarantee of the same candidate count, API availability, model quota, or runtime on future searches.

8. Limitations and known risks:

| Limitation | Impact / interpretation |
|---|---|
| Data coverage and freshness | Crustdata results depend on profile availability, completeness, update frequency, and the fields exposed by the API. A missing skill or job entry may simply be absent from the profile. |
| Search-filter expressiveness | The implemented discovery filters prioritise job-title and country-level search. City-level relevance and technical skill checks are handled after retrieval because |

| | |
|---|---|
| | field/operator support can vary. This can return candidates who later fail location or skill checks, and may miss relevant profiles. |
| Finite search window | Search limits, page limits, and the strict/relaxed/adjacent strategy constrain the candidate pool. The shortlist is not an exhaustive census of the labour market. |
| Enrichment confidence threshold | The client accepts the best enrichment match only when confidence_score is at least 0.8. This reduces weak matches but can leave some valid profiles unenriched. |
| LLM quality and quotas | LLM output can be malformed, inconsistent, or overconfident. Provider quotas, rate limits, token budgets, and model availability can interrupt a run. Validation reduces risk but does not eliminate it. |
| Claim-level rather than ground-truth verification | The verifier checks whether profile evidence supports a claim; it cannot independently establish that the candidate's profile is truthful or current. |
| Scoring is criterion-dependent | Weights are configurable judgement calls. Scores measure evidence against the supplied criteria, not job performance, probability of success, or a hiring recommendation. |
| Timeline inference | Employment dates may be incomplete, month-only, overlapping, or inconsistent. Derived gaps and tenure summaries should be checked with candidates and should not be treated as misconduct. |
| No outreach or workflow integration | The current pipeline produces a research shortlist. It does not send emails, contact candidates, schedule interviews, or integrate a complete applicant-tracking workflow. |
| Privacy and compliance | Candidate profiles contain personal data. Access, retention, sharing, deletion, lawful basis, and regional privacy requirements need organisational controls beyond this prototype. |

9. Future scope and improvement roadmap:

| Roadmap area | Potential enhancement |
|---|---|
| Priority 1 — Evidence quality | Add canonical skill dictionaries and synonym handling; preserve exact source paths in every report row; show evidence provenance and confidence; automatically detect duplicate claims and contradictions across criteria. |
| Priority 2 — Search quality | Validate and expand Crustdata filters; add configurable city aliases, experience ranges, company/industry filters, and query expansion; measure precision and recall against a manually labelled benchmark. |
| Priority 3 — Reliability and cost | Add provider fallback across multiple Groq keys/models with safe quota handling; centralise retry policy; record finish_reason/token usage; introduce cache TTL and cache invalidation; support resumable runs. |
| Priority 4 — Evaluation | Create a labelled candidate/criterion evaluation set; measure claim precision, evidence-path accuracy, verifier agreement, ranking stability, and false-positive rate; add integration tests with mocked API responses. |
| Priority 5 — Reporting | Generate PDF/DOCX directly from the pipeline; add executive summary, criterion heatmap, evidence coverage, rejected-claim appendix, and recruiter notes. |
| Priority 6 — Recruiter workflow | Add a simple web UI, saved searches, review/override controls, audit logs, and export to an ATS only after explicit human approval. |
| Priority 7 — Governance | Implement data minimisation, retention schedules, access control, encryption, deletion workflows, bias testing, and documented human oversight. |

10. Responsible use and governance:

- Use job-related criteria that are consistently applied across candidates.
- Do not use protected characteristics or proxies as ranking features.
- Keep a qualified recruiter responsible for interpreting results and making decisions.
- Verify important claims, dates, tools, project ownership, and location directly with candidates.
- Treat missing profile information as unknown rather than as a negative judgement.

- Restrict access to candidate data, avoid unnecessary exports, and follow applicable privacy and retention rules.
- Do not use generated outreach drafts without human review; the current system does not send them.

11. Project structure:

```
CrustData_Hackathon/
├── pipeline.py              # End-to-end orchestration and report generation
├── crustdata.py             # Crustdata search/enrichment API client
├── llm.py                   # LLM provider, parsing, claim generation, verification
├── prompts.py               # Prompt definitions
├── schemas.py               # Pydantic data models
├── derived.py               # Deterministic derived profile features
├── mock_data.py             # Mock/fallback candidate source
├── requirements.txt         # Python dependencies
├── .env.example             # Configuration template (no real secrets)
├── .env                     # Local secrets; never commit
├── tests/
│   ├── test_core.py
│   ├── test_scoring_regression.py
│   └── test_skill_taxonomy.py
├── .cache/                  # Crustdata response cache; do not commit
├── .llm_cache/              # LLM response cache; do not commit
└── final_recruitment_report.md
```

12. Conclusion:

The project demonstrates a practical evidence-first approach to AI-assisted candidate research. Its key contribution is not simply using an LLM to rank profiles; it combines structured search, profile enrichment, exact-field evidence validation, deterministic semiconductor skill rules, independent claim verification, transparent scoring, and human review. The demonstrated pipeline completed a five-candidate live run and passed 23 automated tests in the recorded environment.

The most valuable next steps are improving source-path traceability, broadening and measuring search quality, hardening provider/quota handling, and building a labelled evaluation benchmark. Any expansion toward automated recruiter workflows should preserve human approval, auditability, and candidate-data safeguards.

Appendix A — Quick-start command sequence

```
cd /mnt/c/Users/Lenovo/CrustData_Hackathon
source .venv/bin/activate
set -a
source .env
set +a
python -m compileall -q .
python -m pytest tests/ -v
python pipeline.py "Find a senior ASIC Physical Design Engineer in Bengaluru,
India. Must-have: professional experience in ASIC physical design. Nice-to-have:
experience with Synopsys tools, timing analysis, floorplanning, placement and
routing." --top 5 --out final_recruitment_report.md
```

Before sharing the project, verify that .env, cache directories, virtual environments, API responses, and unnecessary personal candidate data are excluded from the repository and submission package.